

FOSS Comparison with Examples

Compare Free Software and Open Source Software by applying their principles to real software projects give answer in 10 mark

Here is a **10-mark, exam-ready answer with clear comparison + real software project examples:**

Comparison of Free Software and Open Source Software (with Real Projects)

Free Software and Open Source Software (FOSS) both promote access to source code, but they differ in **philosophy, principles, and focus**. The differences can be clearly understood by applying their principles to real software projects.

1. Philosophy

- **Free Software** focuses on **user freedom and ethics**. “Free” means freedom, not price.
- **Open Source Software** focuses on **practical benefits**, such as better development and collaboration.

 *Example:*

- **GNU/Linux (Free Software):** Emphasizes user freedom and rights.
 - **Android (Open Source):** Focuses on innovation and wide industry adoption.
-

2. Core Principles

- **Free Software** follows four freedoms:
Run, study, modify, and distribute software.
- **Open Source Software** follows principles like transparency, collaboration, and peer review.

 *Example:*

- **LibreOffice (Free Software):** Users can modify and redistribute freely.
- **Apache OpenOffice (Open Source):** Developed openly by the community.

3. Licensing

- **Free Software** commonly uses **GNU GPL**, which enforces freedom through copyleft.
- **Open Source Software** may use permissive licenses like **MIT, BSD, Apache**.

 *Example:*

- **Linux Kernel (GPL):** Modifications must remain open.
 - **Node.js (MIT):** Allows reuse in proprietary products.
-

4. Commercial Usage

- **Free Software** restricts making proprietary derivatives.
- **Open Source Software** allows commercial and closed-source use.

 *Example:*

- **GIMP (Free Software):** Always remains free.
 - **React.js (Open Source):** Used freely in commercial apps.
-

5. Community and Development

- **Free Software** is driven by ideological communities.
- **Open Source Software** often has corporate-backed development.

 *Example:*

- **Debian (Free Software):** Community and freedom-focused.
 - **Kubernetes (Open Source):** Backed by Google and CNCF.
-

6. Security and Transparency

- Both allow code inspection, but **open source** emphasizes faster bug fixing.
- **Free Software** emphasizes trust and control.

 *Example:*

- **GNU Emacs:** Fully user-controlled.
 - **Mozilla Firefox:** Open development and rapid updates.
-

7. User Rights

- **Free Software:** User rights are mandatory and protected.
- **Open Source Software:** Rights depend on the license chosen.

 *Example:*

- **FreeBSD (Open Source):** Can be used in closed systems.
 - **GNU utilities:** Must remain free.
-

8. Adoption in Industry

- **Free Software** is preferred in education and activism.
- **Open Source Software** dominates enterprise systems.

 *Example:*

- **GNU tools in universities.**
 - **Docker, Git, MySQL in companies.**
-

9. Modification and Redistribution

- **Free Software:** Modified versions must stay free.
- **Open Source Software:** Redistribution rules are flexible.

 *Example:*

- **Linux (GPL):** Forces openness.
 - **TensorFlow (Apache License):** Flexible reuse.
-

10. Summary

- **Free Software** prioritizes **freedom and ethics**.
- **Open Source Software** prioritizes **efficiency and collaboration**.

 *Real-world impact:*

Both models power modern software ecosystems, from **Linux servers** to **Android phones** and **cloud platforms**.

Conclusion

While Free Software and Open Source Software share common goals, **Free Software focuses on moral freedom**, whereas **Open Source focuses on practical development advantages**. Real projects like **Linux, Android, LibreOffice, and Kubernetes** clearly demonstrate these differences.

✔ Suitable for 10 marks

If you want, I can also provide a **table format, short answer, or diagram-based comparison**.

table format with 15 points

Here is a **clear, exam-oriented table comparison with 15 points**, suitable for **10–15 marks**:

Comparison between Free Software and Open Source Software

No.	Free Software	Open Source Software
1	Focuses on user freedom and ethics	Focuses on practical development benefits
2	"Free" means freedom , not cost	Source code is open, cost may vary
3	Promoted by Free Software Foundation (FSF)	Promoted by Open Source Initiative (OSI)
4	Emphasizes four freedoms (run, study, modify, share)	Emphasizes collaboration and transparency
5	Uses copyleft licenses like GNU GPL	Uses permissive licenses like MIT, BSD, Apache
6	Modified versions must remain free	Modified versions can be closed-source
7	Restricts proprietary derivatives	Allows commercial and proprietary usage

No.	Free Software	Open Source Software
8	Driven mainly by community and ideology	Often driven by companies and communities
9	Prioritizes user rights and control	Prioritizes efficiency and innovation
10	Less flexible for businesses	Highly flexible for enterprise adoption
11	Strong focus on software freedom movement	Strong focus on industry adoption
12	Example: GNU/Linux, LibreOffice, GIMP	Example: Android, React, Node.js
13	Encourages social responsibility	Encourages faster development cycles
14	Changes must be shared back with the community	Sharing changes is optional
15	Popular in education and activism	Popular in startups and large enterprises

Conclusion

Free Software ensures **freedom and ethical use**, while Open Source Software ensures **innovation, flexibility, and wide adoption**. Both models play a vital role in modern software development.

If you want, I can also **simplify the language, add real-project mapping row-wise, or convert this into a handwritten-exam friendly version**.

Analyse a real-world software project and identify how a FOSS licensing model is implemented. give answer in 10 mark

Below is a **10-mark analytical answer** using a **real-world software project** and clearly explaining how the **FOSS licensing model** is implemented.

Analysis of a Real-World Software Project Using FOSS Licensing

Case Study: Linux Kernel (GNU GPL v2)

The **Linux Kernel** is one of the most successful real-world examples of a **Free and Open Source Software (FOSS)** project. It is licensed under the **GNU General Public License version 2 (GPLv2)**, which strongly enforces the principles of FOSS.

1. Project Overview

Linux is an open-source operating system kernel developed by **Linus Torvalds** and a global community. It is used in servers, desktops, Android devices, cloud platforms, and embedded systems.

2. FOSS License Used

- **License:** GNU General Public License v2 (GPLv2)
- **Type:** Copyleft license

This license ensures that Linux always remains **free and open**.

3. Source Code Availability

The complete Linux source code is publicly available.

➡ Anyone can **view, download, and study** the code.

FOSS Principle Applied: Freedom to study how the software works.

4. Right to Modify

Developers can modify the Linux kernel to:

- Add new drivers
- Improve performance

- Customize for embedded or mobile systems

FOSS Principle Applied: Freedom to modify the software.

5. Redistribution Rights

Users are allowed to **redistribute original or modified versions** of Linux.

Condition:

- The redistributed version **must also be licensed under GPL**.

FOSS Principle Applied: Freedom to share.

6. Copyleft Enforcement

The GPL enforces **copyleft**, meaning:

- Any modified version of Linux **must remain open source**.
- No one can convert Linux into proprietary software.

Impact:

This protects community contributions and user freedom.

7. Commercial Use

Companies like **Red Hat, Canonical, and Google** use Linux commercially.

- They sell **support, customization, and services**, not the code itself.

FOSS Model: Free software with paid services.

8. Community and Governance

Linux development is managed by:

- Open collaboration on platforms like Git
- Contributions reviewed publicly
- Decisions guided by maintainers

FOSS Principle Applied: Community-driven development.

9. Legal Protection and Compliance

The GPL license legally ensures:

- License violations can be challenged
- Source code must be provided when distributing binaries

This maintains long-term openness.

10. Real-World Impact

- Powers **Android OS**, cloud servers, supercomputers
 - Encourages innovation while protecting freedom
 - Demonstrates a **successful FOSS licensing implementation**
-

Conclusion

The Linux Kernel effectively implements the **FOSS licensing model** through GPL by guaranteeing user freedoms, enforcing openness through copyleft, supporting commercial ecosystems, and enabling global collaboration. This makes Linux a strong real-world example of how FOSS licensing works in practice.

Perfect for a 10-mark answer

If you want, I can also provide this analysis using **Android, Mozilla Firefox, or MySQL** as the case study.

real-world software project example

Here is a **10-mark answer** analysing **one real-world software project** and explaining how a **FOSS licensing model is implemented**, written in **simple exam-friendly language**.

Real-World Software Project Using FOSS Licensing

Example: Mozilla Firefox (MPL 2.0 License)

Mozilla Firefox is a popular open-source web browser developed by the **Mozilla Foundation**. It is a real-world software project that successfully follows the **Free and Open Source Software (FOSS)** licensing model.

1. Project Overview

Firefox is a cross-platform web browser used on Windows, Linux, macOS, and mobile devices. It is known for privacy, security, and open standards.

2. FOSS License Used

- **License:** Mozilla Public License (MPL) 2.0
- **Type:** Weak copyleft license

The MPL ensures openness while allowing flexibility for developers.

3. Source Code Availability

Firefox's entire source code is publicly available.
Anyone can **view, study, and download** the code.

FOSS Principle: Freedom to study the software.

4. Right to Modify

Developers are free to modify Firefox's code to:

- Add new features
- Improve security
- Customize the browser

FOSS Principle: Freedom to modify.

5. Redistribution Rights

Modified or unmodified versions of Firefox can be redistributed.

Condition:

- Changes made to Firefox source files must remain open under MPL.
-

6. Copyleft Implementation

MPL uses **file-level copyleft**:

- Modified files must be open source
- Other files can be proprietary

This balances freedom and business needs.

7. Commercial Usage

Companies can:

- Use Firefox code in commercial products
- Add proprietary components alongside open files

This encourages industry adoption.

8. Community Participation

Firefox development involves:

- Global contributors
- Open bug tracking
- Public code reviews

This reflects open collaboration.

9. Legal and Ethical Protection

The MPL legally ensures:

- User freedom
 - Transparency
 - Protection against closed-source misuse
-

10. Real-World Impact

- Used by millions of users worldwide
 - Promotes open web standards
 - Competes with proprietary browsers
-

Conclusion

Mozilla Firefox is a strong real-world example of a **FOSS-licensed project**. By using the **MPL 2.0 license**, it successfully combines user freedom, open development, and commercial flexibility, proving the effectiveness of the FOSS licensing model.

If you want, I can also give the same answer using **Android, Linux, MySQL**, or **Apache HTTP Server** as the example.

Illustrate the steps involved in setting up a dual-boot system with Linux and another OS give answer in 10 marks

Below is a **clear, step-by-step explanation suitable for a 10-mark answer**, using simple and exam-friendly language.

Steps Involved in Setting Up a Dual-Boot System (Linux and Another OS)

A **dual-boot system** allows a computer to run **two operating systems** (for example, **Windows and Linux**) on the same machine. The user can choose which OS to use at startup.

1. Check System Requirements

Before installation:

- Ensure the system meets Linux requirements
 - Check CPU architecture (64-bit)
 - Ensure sufficient disk space (minimum 20–30 GB for Linux)
-

2. Backup Important Data

- Take a full backup of important files from the existing OS
 - This prevents data loss in case of installation errors
-

3. Download Linux Distribution

- Download a Linux ISO file (e.g., **Ubuntu**, **Fedora**, **Linux Mint**)
 - Choose the correct version based on system architecture
-

4. Create Bootable Installation Media

- Use tools like **Rufus**, **Balena Etcher**, or **UNetbootin**
 - Create a bootable USB drive using the Linux ISO file
-

5. Prepare Disk Space

- Open disk management in the existing OS (e.g., Windows Disk Management)
 - Shrink the current partition to create **unallocated space** for Linux
-

6. Change BIOS/UEFI Settings

- Restart the system and enter BIOS/UEFI setup
 - Set USB as the first boot device
 - Disable Secure Boot if required
-

7. Boot from Installation Media

- Insert the bootable USB drive
 - Restart the system and select "Boot from USB"
 - Choose **Install Linux**
-

8. Select Installation Type

- Choose "**Install Linux alongside existing OS**" (recommended)
- or*

- Manually partition disk (/ , swap, /home)
-

9. Install GRUB Boot Loader

- The **GRUB boot loader** is installed automatically
 - It allows users to select **Linux or the other OS** at startup
-

10. Complete Installation and Restart

- Finish installation and remove USB drive
 - Restart the system
 - GRUB menu appears, allowing OS selection
-

Conclusion

Setting up a dual-boot system involves **disk preparation, Linux installation, and bootloader configuration**. It allows users to use both operating systems efficiently on the same hardware.

Perfect for 10 marks

If you want, I can also provide a **diagram-based explanation, short notes version, or Linux-Ubuntu specific steps**.

example

Here is a **clear real-world example** written for a **10-mark answer**, explaining **dual-boot setup using Windows 10 and Ubuntu Linux**.

Example: Setting Up a Dual-Boot System with Windows 10 and Ubuntu Linux

A dual-boot system allows a user to run **Windows 10 and Ubuntu Linux** on the same computer and choose the operating system during startup.

1. Existing System

- Computer already has **Windows 10** installed
 - Single hard disk with enough free space
-

2. Backup Data

- Important files from Windows are backed up
 - This prevents data loss during partitioning
-

3. Download Ubuntu ISO

- Ubuntu 22.04 LTS ISO file is downloaded from the official Ubuntu website
 - 64-bit version is selected
-

4. Create Bootable USB

- A USB drive (8 GB) is used
 - **Rufus** tool is used to create a bootable Ubuntu USB from the ISO file
-

5. Create Free Disk Space

- Windows **Disk Management** tool is opened
 - The main Windows partition is shrunk
 - Around **30 GB unallocated space** is created for Ubuntu
-

6. Change BIOS/UEFI Settings

- System is restarted and BIOS is opened
 - Boot mode set to **UEFI**
 - USB drive set as first boot device
 - Secure Boot disabled (if required)
-

7. Boot from Ubuntu USB

- System boots from the USB drive

- “Try Ubuntu / Install Ubuntu” screen appears
 - User selects **Install Ubuntu**
-

8. Installation Type Selection

- Option selected: “**Install Ubuntu alongside Windows Boot Manager**”
 - Ubuntu automatically uses the unallocated space
-

9. GRUB Bootloader Installation

- **GRUB bootloader** is installed during setup
 - GRUB detects both **Windows 10 and Ubuntu**
-

10. Restart and OS Selection

- Installation completes and system restarts
 - GRUB menu appears
 - User can choose **Ubuntu or Windows 10** at startup
-

Conclusion

This example shows how **Windows 10 and Ubuntu Linux** can be installed on the same system using a dual-boot setup. It allows users to enjoy **Windows applications** and **Linux open-source features** on one machine efficiently.

✅ Ideal for 10 marks

If you want, I can also give a **diagram-based example, Linux Mint + Windows**, or a **shorter exam version**.

Demonstrate Linux process management using system calls such as `fork()`, `exec()`, and `wait()`

Below is a **10-mark, exam-oriented demonstration** of Linux process management using the system calls **fork()**, **exec()**, and **wait()**, with **clear explanation** and **example code**.

Linux Process Management Using **fork()**, **exec()**, and **wait()**

In Linux, **process management** is handled using system calls that allow a program to create, execute, and synchronize processes. The most commonly used system calls are **fork()**, **exec()**, and **wait()**.

1. **fork()** – Creating a New Process

- `fork()` is used to **create a new process**
- The new process is called the **child process**
- The original process is called the **parent process**

Behavior:

- Returns **0** to the child process
- Returns **child PID** to the parent process
- Returns **-1** if creation fails

 **Purpose:** Process creation

2. **exec()** – Executing a New Program

- `exec()` replaces the **current process image** with a **new program**
- Process ID (PID) remains the same
- Common variants: `execl()`, `execv()`, `execvp()`

 **Purpose:** Load and execute a new program

3. **wait()** – Process Synchronization

- `wait()` makes the parent process **wait until the child finishes execution**
- Prevents **zombie processes**
- Returns the PID of the terminated child

 **Purpose:** Parent-child synchronization

Example Program Demonstrating fork(), exec(), and wait()

C

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork(); // Create a new process

    if (pid < 0) {
        // Fork failed
        printf("Process creation failed\n");
    }
    else if (pid == 0) {
        // Child process
        printf("Child Process: PID = %d\n", getpid());
        execl("/bin/ls", "ls", NULL); // Execute ls command
    }
    else {
        // Parent process
        wait(NULL); // Wait for child to finish
        printf("Parent Process: Child execution completed\n");
    }

    return 0;
}
```

Explanation of Program Execution

Step 1: fork()

- Parent process creates a child process
- Both processes execute from the same point

Step 2: Child Process

- `pid == 0` → child process

- Executes `exec1()` to run the `ls` command
- Child process image is replaced by `ls`

Step 3: Parent Process

- Parent waits using `wait(NULL)`
- After child completes, parent resumes execution

Process Flow Diagram (Textual)

bash

Parent Process

|

fork()

|

Child Process ----> `exec(ls)`

|

Parent waits ----> `wait()`

Advantages of Using These System Calls

- Efficient **process creation**
- Supports **multitasking**
- Ensures proper **process synchronization**
- Prevents **zombie processes**

Conclusion

Linux process management is effectively handled using:

- **fork()** for creating processes
- **exec()** for executing new programs
- **wait()** for synchronization

Together, these system calls form the foundation of **multitasking and process control** in **Linux operating systems**.

✓ Perfect for 10 marks

If you want, I can also provide:

- Short-answer version
- Diagram-based explanation
- Output screenshot explanation
- Real-time shell command comparison

Output screenshot explanation

Below is a clear “output screenshot explanation” for the `fork()`, `exec()`, and `wait()` program, written in exam-friendly language (10-mark standard).

Output Screenshot Explanation (`fork()`, `exec()`, `wait()`)

Sample Output (as seen on terminal)

yaml

Child Process: PID = 2456

file1.txt

file2.c

a.out

Parent Process: Child execution completed

Step-by-Step Explanation of the Output

1. Child Process Message

yaml

Child Process: PID = 2456

- This line is printed by the **child process**.
- The PID (Process ID) is **unique** and assigned by the OS.
- This confirms that `fork()` successfully created a child process.
- `fork()` returns **0** in the child, so this block executes.

2. Output of exec() – ls Command

```
csharp
```

```
file1.txt
```

```
file2.c
```

```
a.out
```

- These lines are the **output of the** `ls` **command**.
- The child process calls:

```
c
```

```
exec("/bin/ls", "ls", NULL);
```

- `exec()` **replaces the child process program** with `/bin/ls`.
- After `exec()`, the original child code **does not continue**.
- The directory contents are displayed on the screen.

3. Parent Process Message

```
arduino
```

```
Parent Process: Child execution completed
```

- This line is printed by the **parent process**.
- The parent executes `wait(NULL)`, so:
 - It **waits until the child finishes**
 - It prevents creation of a **zombie process**
- Once the child terminates, the parent resumes and prints this message.

Execution Order as Seen in Screenshot

1. `fork()` creates child process
2. Child prints its PID
3. Child executes `ls` using `exec()`
4. Parent waits using `wait()`
5. Parent prints completion message

This confirms **proper process synchronization**.

Key Observations from Output

- Child process executes first
 - `exec()` output appears before parent message
 - Parent output appears **only after child finishes**
 - Demonstrates **process creation, execution, and synchronization**
-

Conclusion

The output screenshot clearly demonstrates Linux process management:

- `fork()` creates a child process
- `exec()` runs a new program (`ls`) in the child
- `wait()` ensures the parent waits for child completion

This confirms correct implementation of **Linux multitasking and process control**.

✓ Perfect for 10-mark explanation

If you want, I can also provide:

- **Annotated screenshot labels**
- **Viva questions & answers**
- **One-page exam summary**
- **Expected output variations**

Write a Perl program that performs the following tasks:

10

1. Declare an array of 12 integers representing monthly sales figures.

K3

CO2

2. Use a loop to iterate through the array.

3. For each month, classify the sales figure using if-elsif-else as:

High (greater than 1000)

Medium (between 500 and 1000, inclusive)

Low (less than 500)

4. Print each month's sales along with its classification.
 5. At the end, display the total number of months in each category.
- give as a separate code each with the respective output

Below is a **complete, exam-ready Perl solution**.

I am giving **separate sections** for **code** and **output**, clearly labeled, as requested.

✓ Perl Program: Monthly Sales Classification

◆ Task Covered

- Array of 12 monthly sales values
- Loop through array
- Classify sales using `if-elsif-else`
- Print month-wise classification
- Count High / Medium / Low months



Perl Code

```
perl

#!/usr/bin/perl
use strict;
use warnings;

# 1. Declare an array of 12 integers (monthly sales)
my @sales = (1200, 450, 800, 1500, 300, 950, 1100, 700, 400, 1300, 600, 200);

# Counters for categories
my $high = 0;
my $medium = 0;
my $low = 0;

# Month counter
my $month = 1;
```

2. Loop through the array

```
foreach my $sale (@sales) {
```

3. Classify sales using if-elsif-else

```
if ($sale > 1000) {
```

```
    print "Month $month: Sales = $sale -> High\n";
```

```
    $high++;
```

```
}
```

```
elsif ($sale >= 500 && $sale <= 1000) {
```

```
    print "Month $month: Sales = $sale -> Medium\n";
```

```
    $medium++;
```

```
}
```

```
else {
```

```
    print "Month $month: Sales = $sale -> Low\n";
```

```
    $low++;
```

```
}
```

```
    $month++;
```

```
}
```

5. Display total count of each category

```
print "\nSummary:\n";
```

```
print "High sales months  : $high\n";
```

```
print "Medium sales months : $medium\n";
```

```
print "Low sales months   : $low\n";
```



Output

text

Month 1: Sales = 1200 -> High

Month 2: Sales = 450 -> Low

Month 3: Sales = 800 -> Medium

Month 4: Sales = 1500 -> High

Month 5: Sales = 300 -> Low

Month 6: Sales = 950 -> Medium

Month 7: Sales = 1100 -> High

Month 8: Sales = 700 -> Medium

Month 9: Sales = 400 -> Low

Month 10: Sales = 1300 -> High

Month 11: Sales = 600 -> Medium

Month 12: Sales = 200 -> Low

Summary:

High sales months : 4

Medium sales months : 4

Low sales months : 4

Explanation (for examiner)

- `@sales` stores monthly sales data
- `foreach` loop iterates through each month
- `if-elsif-else` classifies sales as High / Medium / Low
- Counters track category totals
- Final summary prints category counts

Perfect for a 10-mark question

If you want, I can also provide:

- **Shorter version (6–8 marks)**
- **User input version**
- **Different sales values**
- **Flowchart or dry run**

Develop a Perl script using scalars, arrays, and hashes for a real-time application.

Below is a **10-mark, exam-ready Perl script** that uses **scalars, arrays, and hashes** in a **real-time application scenario**, with **clear code and output**, suitable for university exams.

Real-Time Application: Student Marks Management System

This script manages student marks, classifies results, and displays a summary using **scalars, arrays, and hashes**.



Perl Script

```
perl

#!/usr/bin/perl
use strict;
use warnings;

# ----- Scalars -----
my $pass_mark = 50;    # Passing mark
my $total_students = 0; # Counter for students

# ----- Array -----
my @students = ("Arun", "Bala", "Charan", "Divya");

# ----- Hash -----
my %marks = (
    "Arun" => 78,
    "Bala" => 45,
    "Charan" => 88,
    "Divya" => 62
);

print "Student Result Report\n";
print "-----\n";

# ----- Processing -----
foreach my $name (@students) {
    my $score = $marks{$name};
    my $result;

    if ($score >= $pass_mark) {
        $result = "PASS";
    } else {
        $result = "FAIL";
    }

    print "Name: $name | Marks: $score | Result: $result\n";
    $total_students++;
}
```

----- Summary -----

```
print "\nTotal Students: $total_students\n";
```

Sample Output

text

Student Result Report

Name: Arun | Marks: 78 | Result: PASS

Name: Bala | Marks: 45 | Result: FAIL

Name: Charan | Marks: 88 | Result: PASS

Name: Divya | Marks: 62 | Result: PASS

Total Students: 4

Explanation (Key Points for 10 Marks)

1. Scalars

- `$pass_mark` stores the minimum marks required to pass
- `$total_students` counts total students processed

2. Arrays

- `@students` stores the list of student names

3. Hashes

- `%marks` stores student names as **keys** and marks as **values**

4. Real-Time Usage

- Used in **schools/colleges** for result processing
- Demonstrates structured data handling

5. Logic Used

- Loop (`foreach`)
- Conditional statement (`if-else`)
- Data retrieval from hash

✓ Conclusion

This Perl script demonstrates how **scalars, arrays, and hashes** are effectively used together in a **real-world student management system** to store, process, and display data efficiently.

If you want, I can also provide:

- **Employee payroll system**
- **Bank account management**
- **Inventory management**
- **User input-based version**
- **Short exam version (8 marks)**

i) Analyse and implement a perl program to read a series of number a and determine the frequency of even and odd numbers using looping and decision making statements.

Below is a **10-mark, exam-ready answer** with **analysis, Perl code, and output**, clearly structured.

(i) Analysis and Implementation: Frequency of Even and Odd Numbers in Perl

◆ Problem Analysis

The objective is to:

- Read a **series of numbers**
 - Use **looping** to iterate through the numbers
 - Use **decision-making statements** to check whether each number is **even or odd**
 - Count the **frequency** of even and odd numbers
-

◆ Logic Used

1. Store the series of numbers in an **array**

2. Initialize counters for even and odd numbers
 3. Use a **foreach loop** to iterate through the array
 4. Use **if-else decision making**:
 - If number `% 2 == 0` → Even
 - Else → Odd
 5. Display the final count of even and odd numbers
-



Perl Program

```
perl

#!/usr/bin/perl
use strict;
use warnings;

# Array containing a series of numbers
my @numbers = (12, 7, 5, 20, 33, 18, 42, 9, 16, 11);

# Counters
my $even_count = 0;
my $odd_count = 0;

# Looping through the array
foreach my $num (@numbers) {

    # Decision making to check even or odd
    if ($num % 2 == 0) {
        $even_count++;
    } else {
        $odd_count++;
    }
}

# Display results
print "Frequency of Even Numbers: $even_count\n";
print "Frequency of Odd Numbers : $odd_count\n";
```



Output

text

Frequency of Even Numbers: 5

Frequency of Odd Numbers : 5

◆ Explanation (Key Points for Marks)

- **Array** (`@numbers`) stores the input series
- **Loop** (`foreach`) iterates through each number
- **Decision statement** (`if-else`) checks even/odd
- **Counters** maintain frequency
- Efficient and simple logic

✓ Conclusion

This Perl program effectively uses **looping and decision-making statements** to read a series of numbers and determine the **frequency of even and odd numbers**, fulfilling the problem requirements.

📌 If you want, I can also provide:

- **User input version**
- **While-loop version**
- **Dry run explanation**
- **Flowchart**

```
#!/usr/bin/perl
use strict;
use warnings;

# Array containing a series of numbers
my @numbers = (12, 7, 5, 20, 33, 18, 42, 9, 16, 11);

# Counters
my $even_count = 0;
my $odd_count = 0;
```

```
# Looping through the array
foreach my $num (@numbers) {

    # Decision making to check even or odd
    if ($num % 2 == 0) {
        $even_count++;
    } else {
        $odd_count++;
    }
}

# Display results
print "Frequency of Even Numbers: $even_count\n";
print "Frequency of Odd Numbers : $odd_count\n";
explain line by line
```

Here is a **clear, line-by-line explanation** of your Perl program, written in **simple exam-friendly language**.

Line-by-Line Explanation of the Perl Program

```
perl
```

```
#!/usr/bin/perl
```

- This is the **shebang line**.
 - It tells the operating system to use the **Perl interpreter** to run this script.
-

```
perl
```

```
use strict;
```

- Enforces **strict programming rules**.
 - Prevents the use of undeclared variables.
 - Helps avoid logical and spelling errors.
-

```
perl
```

```
use warnings;
```

- Enables warning messages.
 - Alerts the programmer about possible mistakes during execution.
-

```
perl
```

```
# Array containing a series of numbers
```

- This is a **comment**.
 - Used for explanation; it does not affect program execution.
-

```
perl
```

```
my @numbers = (12, 7, 5, 20, 33, 18, 42, 9, 16, 11);
```

- Declares an **array** named `@numbers`.
 - Stores a **series of integers** that will be checked for even or odd.
 - `my` limits the variable scope to this program.
-

```
perl
```

```
# Counters
```

- Comment indicating that counters are declared next.
-

```
perl
```

```
my $even_count = 0;
```

- Declares a **scalar variable** to count even numbers.
 - Initialized to `0`.
-

```
perl
```

```
my $odd_count = 0;
```

- Declares a **scalar variable** to count odd numbers.
 - Initialized to `0`.
-

```
perl
```

```
# Looping through the array
```

- Comment explaining the purpose of the next block.
-

```
perl
```

```
foreach my $num (@numbers) {
```

- `foreach` loop iterates through each element of `@numbers`.
 - Each element is stored one by one in the variable `$num`.
-

```
perl
```

```
# Decision making to check even or odd
```

- Comment describing the logic inside the loop.
-

```
perl
```

```
if ($num % 2 == 0) {
```

- Uses the **modulus operator** (%).
 - If remainder is `0`, the number is **even**.
-

```
perl
```



```
$even_count++;
```

- Increments the even number counter by 1.
-

```
perl
```

```
} else {
```

- Executes when the number is **not even**.
-

```
perl
```

```
$odd_count++;
```

- Increments the odd number counter by 1.
-

```
perl
```

```
}
```

- Ends the `if-else` decision block.
-

```
perl
```

```
}
```

- Ends the `foreach` loop after all numbers are processed.
-

```
perl
```

```
# Display results
```

- Comment indicating output statements.
-

```
perl
```

```
print "Frequency of Even Numbers: $even_count\n";
```

- Prints the total count of even numbers.
 - `\n` moves the cursor to the next line.
-

```
perl
```

```
print "Frequency of Odd Numbers : $odd_count\n";
```

- Prints the total count of odd numbers.
-

✓ Summary

- **Array** stores the numbers
- **Loop** processes each number
- **Decision making** (`if-else`) checks even or odd
- **Counters** keep track of frequency

This program clearly demonstrates **looping and decision-making in Perl**.

If you want, I can also give:

- **Dry run table**
- **Flowchart**
- **Viva questions**
- **User input version**

ii) Write a perl program to generate multiplication table for a given number using loops and apply conditional logic to highlight specific multiples. (For eg: highlight multiples of 5).

Below is a **complete, exam-ready answer** for **part (ii)** with **analysis, Perl code, output, and explanation**, suitable for **10 marks**.

(ii) Perl Program: Multiplication Table with Highlighted Multiples

◆ Problem Analysis

The program should:

1. Read a **given number**
2. Generate its **multiplication table** using a **loop**
3. Use **conditional logic** to **highlight multiples of 5**
4. Display the output clearly



Perl Program

```
perl

#!/usr/bin/perl
use strict;
use warnings;

# Given number
my $num = 7;

print "Multiplication Table for $num\n";
print "-----\n";

# Loop to generate table from 1 to 10
for (my $i = 1; $i <= 10; $i++) {

    my $result = $num * $i;

    # Conditional logic to highlight multiples of 5
    if ($result % 5 == 0) {
        print "$num x $i = $result <-- Multiple of 5\n";
    }
    else {
        print "$num x $i = $result\n";
    }
}
```



Sample Output

text

Multiplication Table for 7

```
-----  
7 x 1 = 7  
7 x 2 = 14  
7 x 3 = 21  
7 x 4 = 28  
7 x 5 = 35 <-- Multiple of 5  
7 x 6 = 42  
7 x 7 = 49  
7 x 8 = 56  
7 x 9 = 63  
7 x 10 = 70 <-- Multiple of 5
```

◆ Explanation (Key Points for Marks)

- `$num` stores the number for which the table is generated
- `for` loop runs from 1 to 10
- Multiplication result is stored in `$result`
- `if` condition checks `($result % 5 == 0)`
- Multiples of 5 are highlighted with a message

✓ Conclusion

This Perl program demonstrates:

- Use of **loops** for repetition
- Use of **conditional statements** for decision making
- Real-time logic to **highlight specific multiples**

If you want, I can also provide:

- **User input version**
- **Different highlight condition (even / prime)**
- **Dry run table**
- **Shorter exam answer version**

